

OPERATIONAL-SCRIPTS

Operational Scripts

Four scripts sit beside the wizard. Three handle what goes wrong *after* installation — a corrupting embedding backend, an index that has to be rebuilt because of it, and the question “is this index actually trustworthy?” The fourth is a repository-hygiene gate.

Script	One line	Mutates anything?
<code>scripts/ollama-vulkan-remediation.sh</code>	Take the GPU out of ollama’s embedding path	Only with <code>--apply / --rollback</code> (both need <code>sudo</code>)
<code>scripts/lumen-reindex.sh</code>	Rebuild a Lumen index, refusing to run on a backend known to corrupt it	Yes — writes the index and a log file
<code>scripts/lumen-index-doctor.sh</code>	Detect silently corrupt embeddings in an existing index	No. Read-only, always
<code>scripts/audit-hardcoded-paths.sh</code>	Fail the build on machine-specific absolute paths	No. Read-only, always

They are deliberately separate from `scripts/setup-agents-wizard.sh`. The wizard **observes and reports**; it never restarts a service, never writes under `/etc`, and never calls `sudo` in the embedding path. Test A41 enforces that. Anything privileged lives here, behind an explicit subcommand, so applying a host-wide change is always a decision someone made on purpose.

Test coverage: group **I** for the first three, group **J** for the fourth. Both are covered in [Test coverage](#) below.

Why these exist at all: on 2026-08-26 this project's Lumen index was found to contain **758 stale duplicate vectors** spanning **695 distinct texts across 55 files**, written under HTTP 200 by an ollama runner using library=Vulkan on an Intel iGPU. Read [INDEX-CORRUPTION-RECONCILIATION.md](#) for the settled forensic verdict and [OLLAMA-REMEDIATION.md](#) for the runbook.

The four failure modes these scripts are built around

All three scripts share one model of the defect. A large embedding batch trips an i915 fence timeout; the kernel abandons the dispatch and **ollama reads the result buffer anyway**. What comes back depends on what was in that buffer:

#	Signature	HTTP	Visible to a per-vector check?
1	{"error": "... unsupported value: NaN"}	500	Yes — loud, impossible to miss
2	All-zero vector	200	Yes — a zero-norm test catches it
3	Runner wedges; every later request returns NaN until the model is unloaded	500	Yes, but usually misattributed to the caller
4	A repeated STALE vector — well-formed, 768 dims, unit norm	200	NO. Invisible to NaN, Inf, all-zero and L2-norm checks alike

Mode 4 is the one that matters. It is what corrupted the index, and it is why every probe in these scripts tests **aggregate distinctness** — N distinct input texts must produce N distinct output vectors — instead of inspecting vectors one at a time. Tests I7 and I8 exist specifically to stop that check being quietly removed.

ollama-vulkan-remediation.sh

`./scripts/ollama-vulkan-remediation.sh [--check | --apply | --verify | --rollback | --help]`

Default is --check — running it with no argument does a read-only diagnosis and nothing else (test I6 asserts the literal case "`${1:---check}`"). You cannot accidentally reconfigure your host by forgetting an argument.

What the fix actually is

One line appended to `/etc/sysconfig/ollama`:

```
GGML_VK_VISIBLE_DEVICES=-1
```

The packaged unit already sources that file optionally (`EnvironmentFile=-`), so nothing packaged is edited and no drop-in is created. After a service restart ollama never discovers the Vulkan device, and its own journal reports `library=cpu`.

OLLAMA_VULKAN=false and OLLAMA_LLM_LIBRARY=cpu are both NO-OPS on this build. They were tested. `OLLAMA_VULKAN` is *already* false in the running service and Vulkan is used anyway. `GGML_VK_VISIBLE_DEVICES=-1` is the only variable that changes the outcome.

The model **name** does not change, which is the whole point: Lumen keys each index database by `project_meta.embedding_model`, so switching `LUMEN_EMBED_MODEL` to a CPU-pinned copy would start a second index from zero and orphan the existing one. Disabling Vulkan daemon-wide keeps the model name *and* the index.

The subcommands

Flag	Privileged?	What it does
<code>--check (default)</code>	no	Reports the backend library, whether <code>/etc/sysconfig/ollama</code> carries the flag, the i915 fault count, and runs the 32-vector batch probe. Writes nothing.

Flag	Privileged?	What it does
--apply	yes — sudo	Appends the flag to /etc/sysconfig/ollama if absent, sudo systemctl restart ollama, sleeps 6 s, then runs --verify.
--verify	no	Asserts library=cpu and a clean batch probe. This is the only subcommand whose exit status is a meaningful pass/fail.
--rollback	yes — sudo	Deletes the flag line out of /etc/sysconfig/ollama, restarts ollama, warns that the corrupting path is back, then runs --check.
--help / -h	no	Prints the script's comment header.

The four things --check reports

1. **Backend library.** Read from ollama's own journal, scanned **from the service's ActiveEnterTimestamp** rather than a fixed tail. library= is logged once at start-up, so a fixed tail on a busy log pushes it out of range and reports UNKNOWN. An unreadable journal is reported as UNKNOWN — never guessed at.
2. **Whether /etc/sysconfig/ollama contains the flag.** Three outcomes: present, file exists without the flag, file absent entirely.
3. **i915 fault count since ollama started.** Deliberately not a 24-hour window: on a host that was just remediated, the last 24 h still contain the pre-fix history, and a 24 h count would cry wolf forever. When the since-start count is 0 and the 24 h count is not, the older number is printed as informational context.
4. **The batch probe.** 32 distinct short functions, each repeated 12×, sent as one /api/embed request. It requires **32 of 32 distinct vectors back**. DEGENERATE n/32 means mode 4. MALFORMED means the response could not be parsed as embeddings at all.

Exit codes

Code	When
0	--check, --rollback, --help, and a --verify / --apply that passed
1	--verify failed the batch probe; or --apply / --rollback could not write /etc/sysconfig/ollama or could not restart the service
2	Unrecognised option

--check does not signal through its exit status. It ends 0 whatever it finds — even `library=Vulkan` with a degenerate probe. Read its output, or use `--verify`, which does fail loudly. Automating on `--check`'s exit code will silently pass on a corrupting host.

What it will not do

- **It will not change `LUMEN_EMBED_MODEL`.** That would orphan your index.
- **It will not repair vectors already written.** It changes what happens next. Everything written while Vulkan was active stays wrong — see `lumen-reindex.sh`.
- **It will not fix the upstream bug** (llama.cpp [#18969](#), [#26044](#); ollama [#13086](#), [#15248](#)). It avoids it. Upgrading ollama does not help.
- **It will not run `--apply` for you.** No subcommand escalates on its own, and the wizard never invokes this script — it only *tells you to*.
- **It will not scope the change to embeddings.** `GGML_VK_VISIBLE_DEVICES=-1` disables Vulkan for **everything** the daemon serves, chat models included. That is the stated price.

When to reach for it

Run `--check` whenever a search returns something that feels wrong, whenever the wizard prints the `library=Vulkan` warning, or as the first step of any embedding investigation. Run `--apply` once, deliberately, when `--check` confirms `library=Vulkan`. Run `--verify` after any ollama upgrade or host reboot — a package update that replaces the unit or the environment file puts you straight back on the corrupting path with no other warning.

lumen-reindex.sh

```
./scripts/lumen-reindex.sh [project-path] [--force] [--allow-gpu]
```

A resilient wrapper around `lumen index`. `project-path` defaults to `$ (pwd)`; a leading `--` argument is recognised as a flag rather than a path, so `./scripts/lumen-reindex.sh --force` targets the current directory as expected.

It refuses to start on a Vulkan backend

This is its defining behaviour and test I11 guards it:

```
REFUSING TO START: ollama reports library=Vulkan.  
That path silently writes stale duplicate vectors. Fix it first:  
./scripts/ollama-vulkan-remediation.sh --apply  
Override with --allow-gpu if you accept the risk.
```

→ **exit 3**. `--allow-gpu` downgrades the refusal to a warning and proceeds.

Caveat, from the code: this script reads the backend library from the **last 400 journal lines** (`journalctl -u ollama -n 400`), not from the service start timestamp the way `ollama-vulkan-remediation.sh` does. `library=` is logged once at start-up, so on a host whose ollama has been up a long time under load, that line has scrolled out and the script logs `backend library UNKNOWN (journal unreadable)` - continuing and runs anyway. **The refusal is best-effort, not a guarantee.** The batch probe below is the check that always runs.

Then it probes for mode 4 before writing anything

Independently of the journal, it sends the same 32-distinct-texts batch and requires 32 distinct vectors. On failure it unloads the model (`ollama stop`) to clear a wedged runner, waits 5 s and retries once. Still failing → **exit 4**, nothing indexed.

Then it loops

Up to `MAX_ROUNDS` (default 40) rounds. Each round:

1. Health-check `/api/embed` with a trivial input; reset the runner if unhealthy.
2. Run `lumen index` (with `-f` on the first round if `--force` was given).

3. `rc=0` → done, exit 0.
4. Otherwise: drop `-f` (a forced rebuild only needs forcing once — later rounds resume), then either reset the runner (if the output mentions `NaN` or `embedding servers exhausted`) or back off 30 s.

Every round is appended to the log **and** echoed to stdout.

--force is not optional after a corruption event

A corrupted file still carries a non-empty content hash in the index, so an **incremental** run treats it as already done and skips it — *forever*. `--force` maps to `lumen index -f`, which re-embeds everything. Tests I12 (the flag really reaches `lumen index -f`) and I13 (the reason is documented in the script) guard this.

Flags, environment and exit codes

Argument	Effect
[project-path]	Project to index. Default <code>\$(pwd)</code> .
<code>--force</code>	Full rebuild — <code>lumen index -f</code> on round 1. Required after corruption.
<code>--allow-gpu</code>	Proceed despite <code>library=Vulkan</code> . Off by default, on purpose.

Unrecognised flags are **silently ignored**; there is no argument validation. A typo like `--forse` gets you an incremental run with no complaint.

Variable	Default	Effect
<code>MAX_ROUNDS</code>	40	Retry rounds before giving up
<code>LUMEN_REINDEX_LOG</code>	<code><project>/lumen-reindex.log</code>	Where the run log is appended
<code>LUMEN_EMBED_MODEL</code>	<code>ordis/jina-embeddings-v2-base-code</code>	Model used by both probes
<code>OLLAMA_HOST</code>	<code>http://localhost:11434</code>	Backend endpoint

Code	Meaning
0	Index completed (lumen index returned 0)
1	Gave up after MAX_ROUNDS rounds
3	Refused to start: backend reports library=Vulkan and --allow-gpu was not given
4	Batch probe still failing after a runner reset — the backend is not safe to index with

What it will not do

- **It will not use sudo.** Test I14 asserts the string is absent from this script and the doctor alike.
- **It will not fix the backend.** It refuses, and points at the remediation script.
- **It will not purge.** It never calls `lumen purge`; nothing here deletes an index.
- **It will not verify the result.** Run `lumen-index-doctor.sh` afterwards — a `lumen index` exit code of 0 says the run finished, not that the vectors are good.
- **It will not install Lumen.** It prepends `~/.local/bin` to `PATH` and assumes the wizard put the launcher there. With `lumen` genuinely missing, every round fails and you wait out 40 rounds of 30 s back-off (~20 minutes) before exit 1.

When to reach for it

After applying the Vulkan remediation (`--force`, mandatory). After `lumen-index-doctor.sh` exits 1 (`--force`, mandatory). And as the general-purpose way to run a long index unattended, because it survives the transient backend faults that make a bare `lumen index` die halfway with Lumen's usage text on screen.

lumen-index-doctor.sh

```
./scripts/lumen-index-doctor.sh [project-path]
```

Read-only inspection of the index a project actually uses. It globs `{LUMEN_STORE:-$HOME/.local/share/lumen}/*/index.db`, opens each `file:...?mode=ro` (test I10), and picks the one whose `project_meta.project_path` resolves to your project. No write is ever issued and no `lumen` subcommand is invoked, so it is safe to run while an index is building.

What it reports

```
index: /home/you/.local/share/lumen/<hash>/index.db
model: ordis/jina-embeddings-v2-base-code
integrity_check: ok
files: 2413 fully indexed, 0 queued placeholders | chunks: 35717
vectors: 35717 total, 34959 distinct
    1 duplicate-vector group(s); 758 vectors (2.12%) are not unique
    largest identical group: 758 copies of ONE vector
per-vector: 0 NaN/Inf, 0 all-zero, 0 off-norm
```

- **integrity_check** — SQLite’s own `PRAGMA integrity_check`.
- **File accounting** — a row in `files` with an empty hash is a queued placeholder, not an indexed file. This is how you tell “2413 files” from “2413 files *done*”.
- **Aggregate distinctness** — the shadow table `vec_chunks_vector_chunks00` is decoded directly as contiguous little-endian float32, 768 dims per vector, and identical byte sequences are counted. **This is the test the earlier forensic audit did not complete**, and the only one that sees failure mode 4.
- **Per-vector checks** — NaN/Inf, all-zero, and L2 norm outside 0.99–1.01, retained because they catch the louder modes. Note these are computed over **distinct** vectors, so the numbers are counts of distinct bad vectors, not of affected chunks.

Exit codes

Code	Meaning
0	Healthy — or “no vectors stored yet”, which also exits 0 with a warning line
1	Corruption found
2	Could not inspect: no index found for that project under the store

What actually sets exit 1 — read this before wiring it into anything:

- any NaN/Inf, all-zero, or off-norm vector; **or**
- a duplicate-vector group whose **largest** member count is ≥ 10 .

A handful of genuinely identical vectors (real duplicated boilerplate) is plausible, so small groups are **printed with a line but do not set the exit code**. If you are gating on this script, gate on the output as well as the status.

Two caveats the code makes plain. The decoder hardcodes `DIM = 768` and the shadow-table name `vec_chunks_vector_chunks00`. If a future Lumen changes either, the embedded Python raises, and an unhandled exception also exits 1 — indistinguishable from “corruption found” unless you read the traceback. Separately, the script initialises a `wrongdim` counter that is never incremented or printed: **wrong-dimension vectors are not actually checked**, despite being one of the columns in the forensic report.

What it will not do

- **It will not write, repair, or purge anything.** It has no remediation path at all.
- **It will not use `sudo`** (test I14).
- **It will not call the embedding backend.** It reads the database only — which is precisely why it is safe to run mid-index.
- **It will not tell you *which files are affected*.** It reports group sizes and percentages. The file-level breakdown lives in INDEX-CORRUPTION-RECONCILIATION.md §5.

When to reach for it

Before trusting a search result you find surprising. After any `lumen-reindex.sh --force`. After `ollama-vulkan-remediation.sh --apply`, to confirm the *index* is clean and not merely the *backend*. And periodically on any host that has ever run embeddings on a GPU — a per-vector health check will keep telling you everything is fine.

audit-hardcoded-paths.sh

```
./scripts/audit-hardcoded-paths.sh                # audit this repository
./scripts/audit-hardcoded-paths.sh /path/to/repo  # audit another
```

```
checkout
./scripts/audit-hardcoded-paths.sh --list          # show the
first 50 files it scans
```

Unrelated to embeddings. It exists because **18 tracked files hardcoded one author's /Volumes/... macOS root**, which on every other checkout pointed at nonexistent directories. The worst case was a deploy script running `set -uo pipefail without -e`: its `cd "$ROOT"` failed silently, the script carried on in the caller's working directory, and it then committed and pushed both site submodules. CI had been papering over the whole class by symlinking that path to the workspace.

What counts as a violation

A machine-specific *home* root, matched by:

```
/Volumes/ | /Users/<letter> | /home/<name>/ | /run/media/<letter>
| /mnt/<name>/
```

Explicitly **not** flagged: `/etc`, `/usr`, `/opt`, `/var`, `/tmp` — standard system locations, not somebody's home directory (test J7). Nor `$HOME` or `~`, which are portable by construction (J5).

Paths must be **derived**, never literal:

```
ROOT="$(cd -- "${dirname -- "${BASH_SOURCE[0]}"}/.." && pwd)"
# bash

pathlib.Path(__file__).resolve().parents[N]
# python

path.resolve(__dirname,
              '..') // node CJS
path.dirname(fileURLToPath(import.meta.url)) //
# node ESM
```

Two exemptions, both principled

- **Comment-only lines are stripped before matching**, per language (# for shell/python/yaml, `//` `*` `/*` for C-family). Documenting the historical bug is not the bug. Test J4 proves it with a file whose only occurrence is inside a comment.
- **.hardcoded-paths-allow** at the repository root lists genuinely unavoidable files, one path per line, with the reason on the preceding # line. Test J6 proves a listed file is really suppressed; allowed files are reported as `allowed`, not `hidden`.

Whole directory trees are skipped as *prose or generated evidence, not source*: docs/, _content*, _analysis, _tests/evidence/, .test-evidence/, .superpowers/, .ashlrcode/ and MANUAL-STEPS.md. Binary extensions are skipped too. **This is why the documentation you are reading may quote a real absolute path without failing the audit.**

Exit codes

Code	Meaning
0	No machine-specific paths (allowed files are reported but do not fail). Also --list.
1	Violations found — the count and up to 3 sample lines per file are printed, followed by the derive-instead-of-writing recipes. Also returned if it cannot cd into the resolved root.
2	The directory passed as \$1 does not exist

What it will not do

- **It will not fix anything.** It reports and exits; there is no --fix.
- **It will not scan untracked files.** The file list comes from git ls-files, so a violation you have not git add-ed yet is invisible to it. Group J's fixtures git add -A for exactly this reason.
- **It will not catch a path assembled at runtime** from fragments — which is precisely how the test suite writes its own bad-path literals, so that the file testing the rule does not violate it.
- **It will not use sudo, or write anything.**

When to reach for it

Before any commit that touches a script with a path in it, and as a CI gate. J8 already runs the rule over this repository's own scripts/*.sh on every test run, with no exemptions — the detector itself is excluded by name rather than exempted, because it necessarily contains the patterns it searches for.

The standard sequence

```
# 1. Is the backend safe?
./scripts/ollama-vulkan-remediation.sh --check

# 2. If it reports library=Vulkan – fix it (sudo, one restart),
    then prove it
./scripts/ollama-vulkan-remediation.sh --apply

# 3. Is the existing index already poisoned?
./scripts/lumen-index-doctor.sh "$PWD"           # exit 1 => yes

# 4. Rebuild. --force is mandatory: corrupted files keep their
    hash and are
#     skipped forever by an incremental run.
./scripts/lumen-reindex.sh "$PWD" --force

# 5. Confirm
./scripts/lumen-index-doctor.sh "$PWD"           # exit 0 => clean
```

Step 3 is the one people skip, and it is the one that matters: the backend being fixed today says nothing about the vectors written last week.

Two delegates the wizard DOES call

The four scripts above are invoked by hand. Two others are **called by the wizard**, because the question they answer is per-host or per-provider and must never be answered from a stored constant:

Script	Wizard step	Contract the wizard relies on
scripts/ollama-tune.sh	Step 7, tune_ollama_step	Default run = report only. Three-valued exit, and the wizard preserves all three: 0 fine (nothing to apply), 1 a real problem / action required — i.e. there IS a recommendation — 2 could not determine. --print-commands prints the exact commands for

Script	Wizard step	Contract the wizard relies on
		this host under the same contract; -- apply applies them (and restarts ollama); -- revert undoes them. Path override: OLLAMA_TUNE_SCRIPT
scripts/verify-provider-ci.sh	Step 9, verify_provider_ci_step	Read-only. Exit 0 = no provider-generated triggering found, 1 = confirmed, 2 = could not determine. Path override: PROVIDER_CI_SCRIPT

1 is the trap. It reads like failure and is not: it is the tuner saying *there is something to do*. An integration that maps every non-zero to “broken” throws the recommendation away and reports a host that needs tuning as one whose state is unknown. That bug existed in the first revision of Step 7, was caught by running it against the real script, and is now pinned by assertions K23–K25.

Neither is required for the wizard to run. If the file is absent, or ollama / gh is absent, the wizard says so, records a manual step, and carries on. It never fills the gap with an example value: a concurrency figure that was not measured on *this* machine is worse than none, and a provider status that was not queried is UNVERIFIED, never a pass. That degradation path is asserted behaviourally by test group K, not by grepping for a filename.

Relationship to the wizard and the rollback manifest

scripts/setup-agents-wizard.sh never calls the four scripts documented above. When it detects library=Vulkan it adds an **ACTION REQUIRED** entry naming the remediation commands, and when it cannot confirm a usable index it adds one naming the reindex and

doctor commands — see [ACTION-REQUIRED.md](#). `audit-hardcoded-paths.sh` is not referenced by the wizard at all; it is a repository gate, enforced by test group J.

None of those four writes to the wizard's rollback manifest. They are outside that transaction, so `scripts/rollback-agents-wizard.sh` will not undo them:

Change	How to undo it
<code>GGML_VK_VISIBLE_DEVICES=-1</code> in <code>/etc/sysconfig/ollama</code>	<code>./scripts/ollama-vulkan-remediation.sh --rollback</code>
A rebuilt Lumen index	<code>lumen purge <path></code> , then <code>re-index</code>
<code><project>/lumen-reindex.log</code>	<code>rm</code> it; nothing reads it back
<i>(nothing)</i> — <code>lumen-index-doctor.sh</code> and <code>audit-hardcoded-paths.sh</code>	Read-only by construction

The two delegates are the exception. `ollama-tune.sh` **is** inside the transaction whenever the wizard applied it: Step 7 writes an `ACTION` row carrying `<resolved path> --revert` plus a `NOTE` row stating that the service restart aborted whatever embedding requests were in flight and that `--revert` cannot bring those back. A report-only run writes neither row, because it changed nothing. `verify-provider-ci.sh` writes no rows at all — it only reads.

Test coverage — groups I and J

`scripts/test-setup-agents-wizard.sh` covers the three embedding scripts in group I:

Test	Asserts
I1–I3	Each script is executable and parses (<code>bash -n</code>) — one record per script
I4, I5	The remediation script offers <code>--check</code> and <code>--rollback</code>
I6	It defaults to the read-only action
I7	

Test	Asserts
	Its probe tests aggregate distinctness , not per-vector health
I8	The doctor checks duplicate-vector groups
I9	The doctor still runs the per-vector NaN / all-zero checks
I10	The doctor opens the database mode=ro
I11	Reindex refuses to start on a Vulkan backend
I12	--force is parsed and actually reaches lumen index -f (behavioural, not a grep)
I13	Reindex documents why --force is required after corruption
I14	Neither reindex nor doctor calls sudo

Fourteen records in total: eleven assertions plus the three-script parse loop.

Group **J** covers the path audit, and does so *behaviourally* — the script is run against throwaway git repositories rather than grepped, so each verdict is proven:

Test	Asserts
J1	The audit script is executable and parses
J2	Exits 0 on a repo with no hardcoded paths
J3	Exits 1 when a machine-specific path is present
J4	A comment describing the historical bug is not a violation
J5	\$HOME and ~ are not flagged
J6	.hardcoded-paths-allow really suppresses a listed file
J7	

Test	Asserts
J8	/etc, /usr, /tmp are not treated as machine-specific This repository's own scripts/ *.sh are clean — no exemptions

Note the test file builds its bad-path literals from **concatenated fragments** ("/Vol""umes/..."), so that the file testing the rule does not itself violate J8.

Known rough edges

Verified against the current sources, and listed here so nobody re-reports them:

- **--help over-prints.** ollama-vulkan-remediation.sh --help runs sed -n '2,40p' over itself, but the comment header ends at line 34 — so the last six lines of “help” are the script's own set -uo pipefail and colour definitions. Harmless, confusing.
- **The applied file is bare.** --apply appends the single line GGML_VK_VISIBLE_DEVICES=-1. The runbook's Step 2 heredoc writes the same line with an explanatory comment block and chmod 0644. If /etc/sysconfig/ollama on your host has no comment header, it was written by the script, not by hand.
- **--check always exits 0** (see above).
- **The doctor does not check dimensions** despite the dead wrongdim counter.
- **lumen-reindex.sh ignores unknown flags** and reads the journal by fixed tail.
- **audit-hardcoded-paths.sh only sees tracked files** (git ls-files), so a violation you have not staged yet does not fail it.

Related

- [OLLAMA-REMEDICATION.md](#) — the full operator runbook, all three tiers, and the resolution record for this host.
- [INDEX-CORRUPTION-RECONCILIATION.md](#) — the settled verdict on what was in the index. **Historical record; do not edit.**
- [OLLAMA-NAN-WEDGE.md](#) — the original evidence and harness for the loud failure modes. **Historical record; do not edit.**

- LUMEN-INDEX-INTEGRITY.md — the earlier audit that concluded “TRUSTWORTHY”. Its measurements are correct; its conclusion is **superseded. Historical record; do not edit.**
- INDEPENDENT-VERIFICATION-2.md — an adversarial pass that put 15 claims about this stack under test. **Historical record; do not edit.**
- ACTION-REQUIRED.md — how the wizard surfaces these scripts.
- TROUBLESHOOTING.md — symptom-first entry points.